Task 1.0: Set up Colab

(Done before recitation!)

Task 1.1: Copy code into a blank notebook

We have given you a short `ipynb` file with code that you will need for this lab. The first part imports packages with commands like this one:

```
import numpy as np
```

Copy this code into your Colab notebook and run the cell.

Why doesn't numpy include a function for RREF?

Row reduction is very useful for learning concepts such as rank, linear independence, and solving systems of equations. However, when computers solve large numerical problems, they usually avoid Gauss–Jordan elimination. Instead they use other matrix factorizations that are faster and more numerically stable when working with floating-point arithmetic.

For example, numerical libraries often rely on methods like

- **LU factorization** for solving systems of equations,
- **QR factorization** for least-squares problems (linear regression), and
- **Singular Value Decomposition (SVD)** for tasks like data analysis and image compression.

These methods are better suited to large matrices and the rounding errors that occur with floating-point numbers. Because of this, `numpy` focuses on providing tools for these widely used numerical algorithms rather than RREF.

In this lab we will implement our own `rref` function. (It's defined for you in the attached code!)

Task 1.2: Getting acquainted with matrices in Python

We use the Python package `numpy` for linear algebra. To define a matrix in `numpy`, we can use the `np.array` function and give the matrix *one row at a time*. For example, to define the matrix

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 4 & 5 & 6 & 7 \\ 7 & 8 & 9 & 10 \end{bmatrix},$$

we can write code like this:

```
A = np.array([[1, 2, 3, 4],
              [4, 5, 6, 7],
              [7, 8, 9, 10]])
```

You could also leave this all on one line, like this:

```
A = np.array([[1, 2, 3, 4], [4, 5, 6, 7], [7, 8, 9, 10]])
```

If you want to print a matrix, you can use the `print` function, but it will not look very nice. Instead, we provide a function `show_matrix` that renders matrices more clearly. Try it on the matrix you just defined.

```
print(A)                # not very nice
show_matrix(A)        # much better!
```

(Any text following a `#` symbol is a comment and is ignored by Python. We use comments to explain what the code does.)

Entries and shape

Warning: Python uses 0-based indexing, which means that the first row and column of a matrix are indexed by 0, not 1. So the entry in the first row and first column of `A` is `A[0,0]`, not `A[1,1]`.

Matrix indices in numpy

Mathematicians usually write a matrix like this:

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix}$$

In `numpy`, indexing starts at 0 instead of 1.
Some examples:

Math notation	`numpy`
$a_{1,1}$	`A[0,0]`
$a_{1,3}$	`A[0,2]`
$a_{2,1}$	`A[1,0]`
$a_{3,2}$	`A[2,1]`

A helpful rule is

$$a_{i,j} = A[i-1,\ j-1].$$

- To refer to an entry of a matrix, we use square brackets. For example, `A[2,3]` gives the entry in the *3rd* row and *4th* column of `A` (remember the 0-indexing).
- To get the number of rows and columns of a matrix, use the `shape` attribute. For example, `A.shape` might return the tuple `(3,4)`, meaning that `A` has 3 rows and 4 columns.
- To get the number of rows, use `A.shape[0]`. To get the number of columns, use `A.shape[1]`.

Rows and columns

To get the second row of a matrix, use `A[2,:]`. To get the third column, use `A[:,3]`.

The output is a one-dimensional array (a vector), not a matrix.

Matrix operations

Unless you have redefined it, `A` should still be the 3×4 matrix we defined at the beginning of this task. If you forget what it is, you can always call `print(A)` or `show_matrix(A)` again.

- To add two matrices, use the `+` operator. Try code like this to see what you get when you add the matrix `A` to itself: `print("A + A:", A + A)`
- To multiply a matrix by a scalar, use the `*` operator. E.g. `3 * A` gives the matrix obtained by multiplying each entry of `A` by 3. Try displaying `3 * A` using either `print` or `show_matrix`.
- To multiply two matrices, use the `@` operator. E.g. `B @ C` gives the matrix obtained by multiplying `B` by `C`.

Try multiplying `A` by itself. Why does it give an error? Now try multiplying `A` by its transpose: `A @ A.T`. What do you get?

Special matrices

`numpy` offers functions to create special matrices. For example, `np.eye(n)` produces the $n \times n$ identity matrix, and `np.zeros((m,n))` produces the $m \times n$ zero matrix.

Task 2: Problems

Problem 1

Define the matrices

$$A = \begin{bmatrix} 2 & 3 & 1 \\ 2 & 1 & 1 \end{bmatrix}, \qquad B = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \end{bmatrix}$$

in `numpy`. Then compute

$$C = 3A + 2B.$$

Display the result using `show_matrix`. (*Hint*: Don't forget to use the `*` operator for scalar multiplication.)

Problem 2

Define two different 4×2 matrices of your choice and add them together using `numpy`. Display the result using `show_matrix`.

Problem 3

Enter a matrix of size 3×6 and compute its reduced row echelon form using the `rref` function. Use the output to determine the rank of the matrix.

Problem 4

Define an invertible 3×3 matrix and a non-invertible 3×3 matrix.

- Use the `rref` function to show that the first matrix is invertible and the second is not.
- Use the inverse function `np.linalg.inv` to compute the inverse of the first matrix.
- What happens when you try to compute the inverse of the second matrix? Why does this give an error?

Problem 5

Use the `rref` function to solve the system

$$\begin{array}{rcrcrcr} 4x_1 & - & 5x_2 & + & 7x_3 & = & 6 \\ -x_1 & + & 3x_2 & - & 8x_3 & = & -8 \\ 7x_1 & - & 5x_2 & & & = & 0 \\ -4x_1 & + & x_2 & + & 2x_3 & = & -7 \end{array}$$

Problem 6

Consider the matrix

$$D = \begin{bmatrix} 1 & 0 & 1 \\ 1 & -1 & 2 \\ 2 & -1 & 3 \end{bmatrix}.$$

- Use `rref` to solve the matrix equation $D\mathbf{x} = \mathbf{0}$.
- What is the `numpy` command for extracting the second column of D? Run this command (remember the 0-indexing).

Task 3: Loading and viewing images

Reading and writing images can be done using these functions:

- `imageio.imread`
- `imageio.imwrite`

Make sure that the file `lab1picture.jpg` is in the same folder as your notebook. Then run this code in a new code block:

```
image = imageio.imread('lab1picture.jpg')
```

Display the picture using this code:

```
plt.figure()
plt.imshow(image)
plt.show()
```

A color image consists of three color channels: **red**, **green**, and **blue**. Thus the variable `image` is actually a three-dimensional array. Use the `shape` attribute (Hint: `image.shape`) to determine the dimensions of the array. The three dimensions represent the rows, columns, and color channels of the image, respectively.

The following code should display only the red channel:

```
r = image[:,:,0]
plt.figure()
plt.imshow(r, cmap='Reds')
plt.show()
```

Can you do the same for the green and blue channels?